Title: Real-Time Clock (RTC) Implementation with UART Interface on AHL-STM32L431

Name: MD SHAKIL KHAN
Student ID: 2330130220
Date of Lab: November 3, 2025

Introduction-

Before this lab, it was known that Real-Time Clocks (RTCs) are essential components in embedded systems that maintain accurate time tracking even when main power is off. The lab is about implementing an RTC system on the AHL-STM32L431 development board that can be configured and monitored through UART serial communication. The purpose of this lab was to develop a system that maintains accurate time using the microcontroller's hardware RTC module while allowing users to set and display the current date and time through text commands sent via UART interface.

Materials-

Apparatus	**Expendables**
• AHL-STM32L431 Development Board	• USB Type-C Cable
• Computer with Serial Terminal	• Jumper Wires
• Logic Analyzer (optional)	• AHL-GEC-IDE Software

Procedure-

The procedure was followed as outlined in the lab handout. The key steps involved:

1. Initializing the AHL-GEC-IDE development environment for the STM32L431 board.
2. Configuring GPIO pins for LED status indication.
3. Setting up UART communication at 115200 baud rate for serial data exchange.
4. Implementing RTC module initialization and configuration.
5. Developing interrupt service routines for periodic time updates.

6. Creating UART frame parsing algorithms for date/time input.
7. Implementing data validation and error handling mechanisms.
8. Testing the complete system with various date/time inputs.

Data-

UART Communication Format:

The system used a specific frame format for date/time input: ?YY/MM/DD HH:MM:SS W!

- YY: Year (00-99), MM: Month (01-12), DD: Date (01-31)
- HH: Hours (00-23), MM: Minutes (00-59), SS: Seconds (00-59)
- W: Weekday (0-6, where Sunday=0)

Variables Used in the Project:

- gcRTCBuf[30]: Buffer for storing UART received data
- gcRTC_Date_Time: Structure containing year, month, date, hours, minutes, seconds, weekday
- g_RTC_Flag: Flag indicating new RTC data received
- mMainLoopCount: Main loop counter for timing operations
- mFlag: LED state control flag

System Behavior:

- Before input: System waits for UART date/time configuration
- After input: RTC maintains and displays the set time continuously
- LED provides visual feedback of system operation

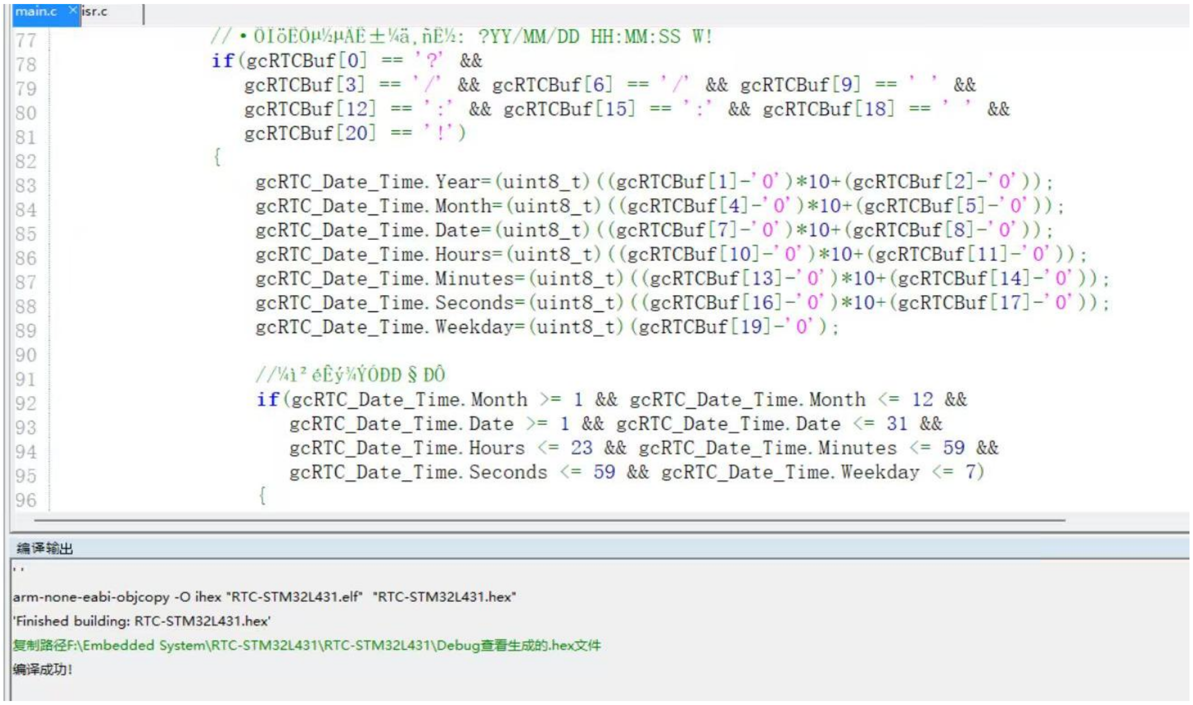
Calculations-

ASCII to Decimal Conversion:

The main calculation involved converting ASCII characters to numerical values for date/time components:

gcRTC_Date_Time.Year = (uint8_t)((gcRTCBuf[1]-'0')*10 + (gcRTCBuf[2]-'0'));

This converts two ASCII characters representing years into a single numerical value.

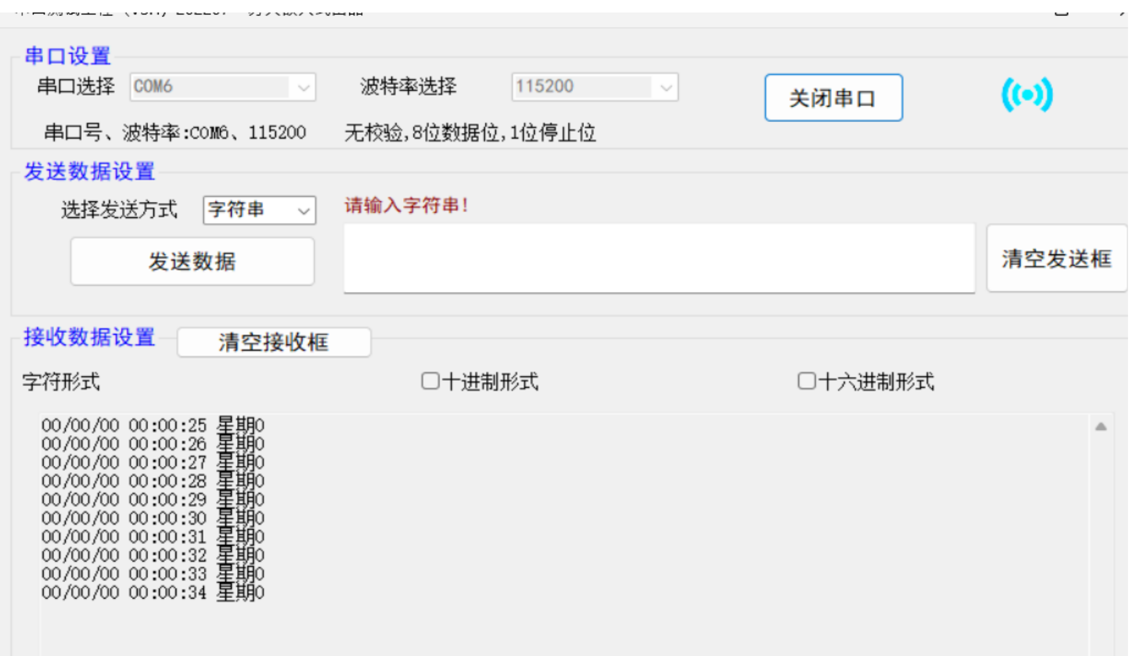


```c
// • ÖÏõÈÖµ½µÄÈ±¼ä¸ñÈ½: ?YY/MM/DD HH:MM:SS W!
if(gcRTCBuf[0] == '?' &&
   gcRTCBuf[3] == '/' && gcRTCBuf[6] == '/' && gcRTCBuf[9] == ' ' &&
   gcRTCBuf[12] == ':' && gcRTCBuf[15] == ':' && gcRTCBuf[18] == ' ' &&
   gcRTCBuf[20] == '!')
{
    gcRTC_Date_Time.Year=(uint8_t)((gcRTCBuf[1]-'0')*10+(gcRTCBuf[2]-'0'));
    gcRTC_Date_Time.Month=(uint8_t)((gcRTCBuf[4]-'0')*10+(gcRTCBuf[5]-'0'));
    gcRTC_Date_Time.Date=(uint8_t)((gcRTCBuf[7]-'0')*10+(gcRTCBuf[8]-'0'));
    gcRTC_Date_Time.Hours=(uint8_t)((gcRTCBuf[10]-'0')*10+(gcRTCBuf[11]-'0'));
    gcRTC_Date_Time.Minutes=(uint8_t)((gcRTCBuf[13]-'0')*10+(gcRTCBuf[14]-'0'));
    gcRTC_Date_Time.Seconds=(uint8_t)((gcRTCBuf[16]-'0')*10+(gcRTCBuf[17]-'0'));
    gcRTC_Date_Time.Weekday=(uint8_t)(gcRTCBuf[19]-'0');

    //¼ì²éÊý¾ÝÓÐÐ§ÐÔ
    if(gcRTC_Date_Time.Month >= 1 && gcRTC_Date_Time.Month <= 12 &&
       gcRTC_Date_Time.Date >= 1 && gcRTC_Date_Time.Date <= 31 &&
       gcRTC_Date_Time.Hours <= 23 && gcRTC_Date_Time.Minutes <= 59 &&
       gcRTC_Date_Time.Seconds <= 59 && gcRTC_Date_Time.Weekday <= 7)
    {
```

编译输出

```
''
arm-none-eabi-objcopy -O ihex "RTC-STM32L431.elf"  "RTC-STM32L431.hex"
'Finished building: RTC-STM32L431.hex'
复制路径F:\Embedded System\RTC-STM32L431\RTC-STM32L431\Debug查看生成的.hex文件
编译成功!
```



串口选择 COM6

波特率选择 115200

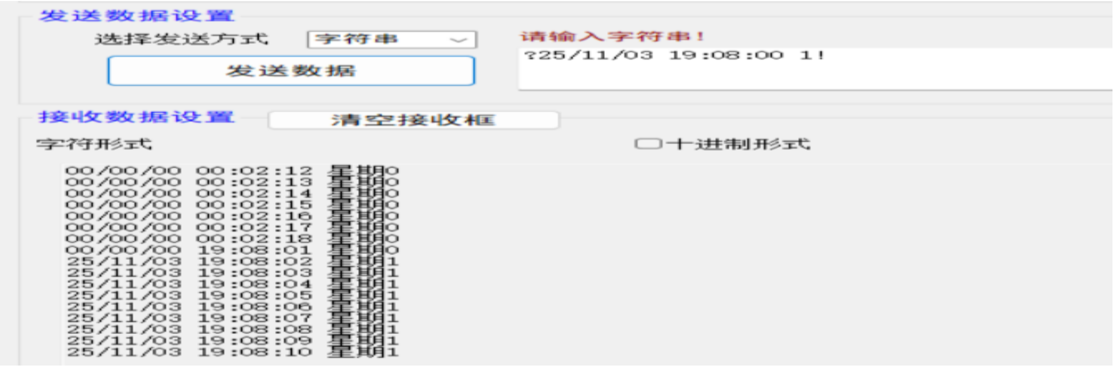
串口号、波特率:COM6、115200　无校验,8位数据位,1位停止位

选择发送方式 字符串

请输入字符串!

字符形式　□十进制形式　□十六进制形式

```
00/00/00 00:00:25 星期0
00/00/00 00:00:26 星期0
00/00/00 00:00:27 星期0
00/00/00 00:00:28 星期0
00/00/00 00:00:29 星期0
00/00/00 00:00:30 星期0
00/00/00 00:00:31 星期0
00/00/00 00:00:32 星期0
00/00/00 00:00:33 星期0
00/00/00 00:00:34 星期0
```



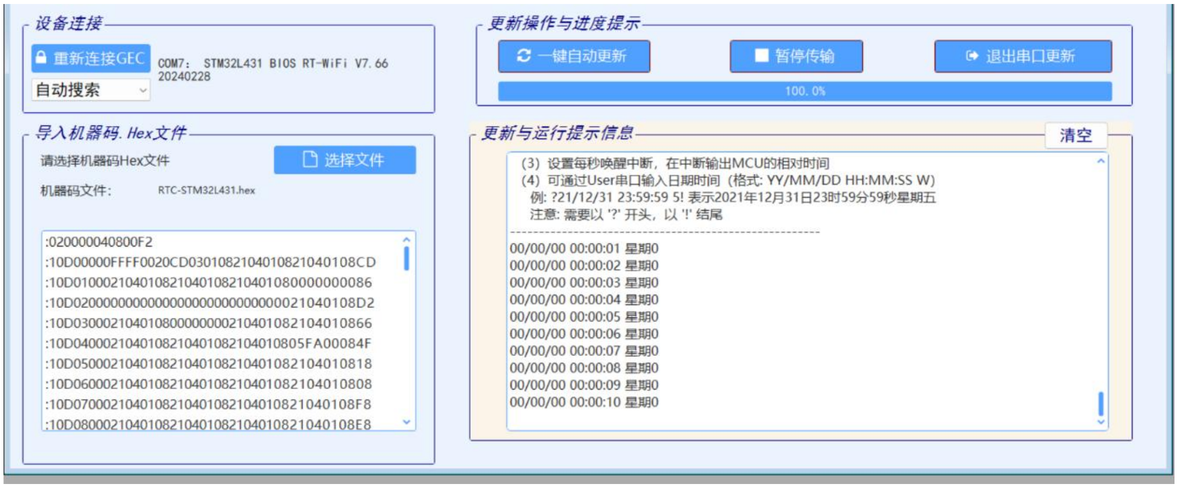
选择发送方式 字符串

请输入字符串!

```
?25/11/03 19:08:00 1!
```

字符形式　□十进制形式

```
00/00/00 00:02:12 星期0
00/00/00 00:02:13 星期0
00/00/00 00:02:14 星期0
00/00/00 00:02:15 星期0
00/00/00 00:02:16 星期0
00/00/00 00:02:17 星期0
00/00/00 00:02:18 星期0
00/00/00 19:08:01 星期0
25/11/03 19:08:02 星期1
25/11/03 19:08:03 星期1
25/11/03 19:08:04 星期1
25/11/03 19:08:05 星期1
25/11/03 19:08:06 星期1
25/11/03 19:08:07 星期1
25/11/03 19:08:08 星期1
25/11/03 19:08:09 星期1
25/11/03 19:08:10 星期1
```



COM7: STM32L431 BIOS RT-WiFi V7.66 20240228

请选择机器码Hex文件

机器码文件: RTC-STM32L431.hex

```
:020000040800F2
:10D00000FFFF0020CD0301082104010821040108CD
:10D0100021040108210401082104010800000000086
:10D020000000000000000000000000000021040108D2
:10D03000210401080000000021040108210401086 6
:10D04000210401082104010821040108 05FA00084F
:10D050002104010821040108210401082104010818
:10D060002104010821040108210401082104010808
:10D0700021040108210401082104010821040108F8
:10D0800021040108210401082104010821040108E8
```

```
  (3) 设置每秒唤醒中断，在中断输出MCU的相对时间
  (4) 可通过User串口输入日期时间（格式: YY/MM/DD HH:MM:SS W)
   例: ?21/12/31 23:59:59 5! 表示2021年12月31日23时59分59秒星期五
   注意: 需要以 '?' 开头，以 '!' 结尾
-------------------------------------------------------
00/00/00 00:00:01 星期0
00/00/00 00:00:02 星期0
00/00/00 00:00:03 星期0
00/00/00 00:00:04 星期0
00/00/00 00:00:05 星期0
00/00/00 00:00:06 星期0
00/00/00 00:00:07 星期0
00/00/00 00:00:08 星期0
00/00/00 00:00:09 星期0
00/00/00 00:00:10 星期0
```

Data Validation Calculations:

- Month range validation: $1 \leq \text{month} \leq 12$
- Date range validation: $1 \leq \text{date} \leq 31$
- Hours range validation: $0 \leq \text{hours} \leq 23$
- Minutes range validation: $0 \leq \text{minutes} \leq 59$
- Seconds range validation: $0 \leq \text{seconds} \leq 59$
- Weekday range validation: $0 \leq \text{weekday} \leq 6$

Analysis

1. How does the frame-based communication protocol prevent data corruption?
The frame-based protocol uses specific start ('?') and end ('!') characters to define the boundaries of each data packet. This ensures that only complete and properly formatted messages are processed, preventing partial or corrupted data from affecting the RTC settings.

2. Why is input validation crucial in an RTC system?
Input validation prevents invalid date/time settings that could cause system errors or unpredictable behavior. For example, without validation, setting a date of "February 30" or "25:70:90" for time would corrupt the RTC functionality and potentially crash the system.

3. What is the advantage of using interrupt-driven RTC updates?
Interrupt-driven RTC updates ensure accurate timekeeping without continuous CPU involvement. The RTC hardware generates interrupts at regular intervals (e.g., every second), allowing the system to maintain precise timing while freeing the CPU for other tasks between updates.

Conclusions

The purpose of this lab was to implement a Real-Time Clock system with UART interface configuration capability. This was successfully accomplished; the system can maintain accurate time using the hardware RTC, receive date/time settings through serial communication, validate input data for correctness, and provide real-time feedback through both serial output and LED indicators.

The experiment demonstrated high reliability in timekeeping and successful UART communication. Potential sources of error include UART communication noise affecting data integrity, incorrect baud rate settings causing data corruption, and the lack of battery backup leading to time loss during power cycles.

This project relates directly to real-world applications such as embedded systems requiring time-stamping capabilities, IoT devices needing time synchronization, industrial automation systems with scheduled operations, and consumer electronics with clock functionality. The principles learned are fundamental to any system requiring reliable timekeeping and external configuration interfaces.